



Lab 10

Implementing Data Encryption Standard (DES)

Name: Syed Saad Imran/Huzefa Soni	Student ID: 09370/08940
-----------------------------------	-------------------------

10.1 Data Encryption Standard

10.1 Objective

The Objectives of this lab are:

- Perform the initial permutation and final permutation.
- Constructing the DES function
- Designing the Key Generator function
- Implementing the Cipher algorithm
- Encrypt data using the Data Encryption Standard (DES) algorithm.
- Validate the DES implementation with various inputs.

10.2 Background/Scenario

The Data Encryption Standard (DES) is a historically significant symmetric-key block cipher encryption algorithm that played a pivotal role in the early days of computer cryptography. Developed by IBM in the 1970s, DES was adopted as a federal standard in the United States and became one of the most widely used encryption schemes in the world for several decades. In this lab, you will implement Data Encryption Standard (DES) using Verilog HDL, and subsequently employ it to encrypt your data. or the submission, ensure that you provide all your modules and results where applicable under the relevant task headings.

Task 1 – Initial and Final Permutation Tables

<i>Initial Permutation</i>	<i>Final Permutation</i>
58 50 42 34 26 18 10 02	40 08 48 16 56 24 64 32
60 52 44 36 28 20 12 04	39 07 47 15 55 23 63 31
62 54 46 38 30 22 14 06	38 06 46 14 54 22 62 30
64 56 48 40 32 24 16 08	37 05 45 13 53 21 61 29
57 49 41 33 25 17 09 01	36 04 44 12 52 20 60 28
59 51 43 35 27 19 11 03	35 03 43 11 51 19 59 27
61 53 45 37 29 21 13 05	34 02 42 10 50 18 58 26
63 55 47 39 31 23 15 07	33 01 41 09 49 17 57 25



Figure 1: Initial and Final Permutation tables

1. Create a module that returns as output the initial permutation table consisting of 64 bits. Refer to Figure 1 for the change in position of each bit.

```
module initial_permutation (
    input wire [63:0] in_data,
    output wire [63:0] out_data
);

assign out_data = {
    in_data[6], in_data[14], in_data[22], in_data[30], in_data[38], in_data[46], in_data[54], in_data[62], // Row 1
    in_data[4], in_data[12], in_data[20], in_data[28], in_data[36], in_data[44], in_data[52], in_data[60], // Row 2
    in_data[2], in_data[10], in_data[18], in_data[26], in_data[34], in_data[42], in_data[50], in_data[58], // Row 3
    in_data[0], in_data[8], in_data[16], in_data[24], in_data[32], in_data[40], in_data[48], in_data[56], // Row 4
    in_data[7], in_data[15], in_data[23], in_data[31], in_data[39], in_data[47], in_data[55], in_data[63], // Row 5
    in_data[5], in_data[13], in_data[21], in_data[29], in_data[37], in_data[45], in_data[53], in_data[61], // Row 6
    in_data[3], in_data[11], in_data[19], in_data[27], in_data[35], in_data[43], in_data[51], in_data[59], // Row 7
    in_data[1], in_data[9], in_data[17], in_data[25], in_data[33], in_data[41], in_data[49], in_data[57] // Row 8
};

endmodule
```

2. Create a module that returns as output the final permutation table consisting of 64 bits. Refer to Figure 1 for the change in position of each bit.

```
module final_permutation (
    input wire [63:0] in_data,
    output wire [63:0] out_data
);

assign out_data = {
    in_data[24], in_data[56], in_data[16], in_data[48], in_data[8], in_data[40], in_data[0], in_data[32], // Row 1
    in_data[25], in_data[57], in_data[17], in_data[49], in_data[9], in_data[41], in_data[1], in_data[33], // Row 2
    in_data[26], in_data[58], in_data[18], in_data[50], in_data[10], in_data[42], in_data[2], in_data[34], // Row 3
    in_data[27], in_data[59], in_data[19], in_data[51], in_data[11], in_data[43], in_data[3], in_data[35], // Row 4
    in_data[28], in_data[60], in_data[20], in_data[52], in_data[12], in_data[44], in_data[4], in_data[36], // Row 5
    in_data[29], in_data[61], in_data[21], in_data[53], in_data[13], in_data[45], in_data[5], in_data[37], // Row 6
    in_data[30], in_data[62], in_data[22], in_data[54], in_data[14], in_data[46], in_data[6], in_data[38], // Row 7
    in_data[31], in_data[63], in_data[23], in_data[55], in_data[15], in_data[47], in_data[7], in_data[39] // Row 8
};

endmodule
```

Task 2 – Constructing the DES function

The DES function applies a 48-bit key to the rightmost 32 bits (RI-1) to produce a 32-bit output. This function is made up of four sections: an expansion P-box, a whitener (that adds key), a group of S-boxes, and a straight P-box

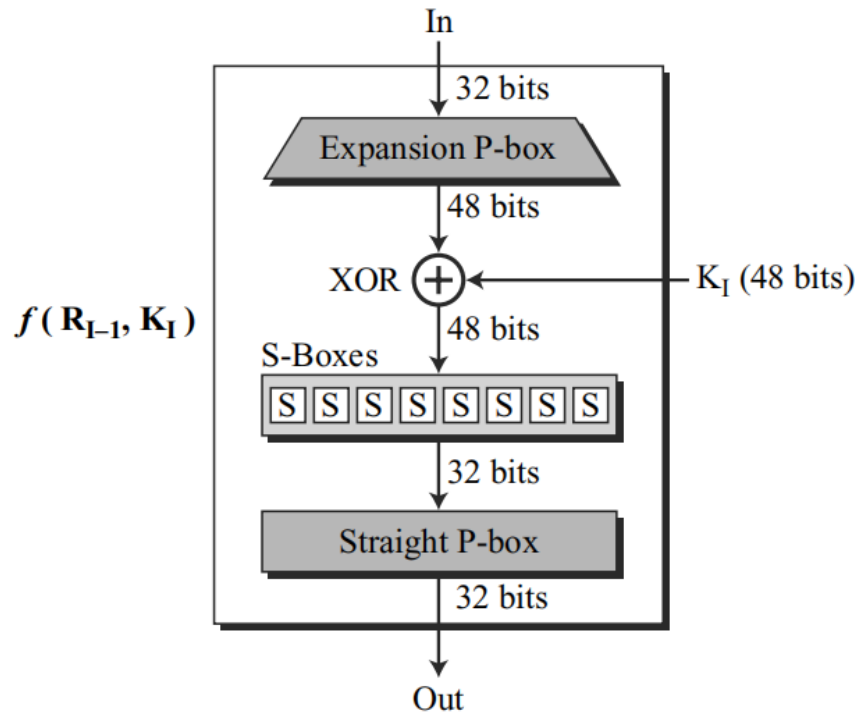


Figure 2: DES function



```
module des_function (  
    input wire [31:0] in_data,  
    input wire [47:0] round_key,  
    output wire [31:0] out_data  
);  
    wire [47:0] expanded_data;  
    wire [47:0] xor_data;  
    wire [31:0] sbox_data;  
    expansion_p_box ep_box (  
        .in_data(in_data),  
        .out_data(expanded_data)  
    );  
  
    whitener_xor xor_step (  
        .expanded_data(expanded_data),  
        .round_key(round_key),  
        .xor_out(xor_data)  
    );  
  
    sboxes_combined s_boxes (  
        .in_data(xor_data),  
        .out_data(sbox_data)  
    );  
  
    straight_permutation sp_box (  
        .in_data(sbox_data),  
        .out_data(out_data)  
    );  
  
endmodule
```

Task 2.1 - Creating the Expansion P-Box Module

Use Figure 3 to create a module that takes as input 32-bit and returns as output a 48-bit text.



32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	01

Figure 3: Expansion P-box Table

```
module expansion_p_box (  
    input wire [31:0] in_data,  
    output wire [47:0] out_data  
);  
  
    assign out_data = (  
        in_data[0], in_data[31], in_data[30], in_data[29], in_data[28], in_data[27], // Row 1  
        in_data[28], in_data[27], in_data[26], in_data[25], in_data[24], in_data[23], // Row 2  
        in_data[24], in_data[23], in_data[22], in_data[21], in_data[20], in_data[19], // Row 3  
        in_data[20], in_data[19], in_data[18], in_data[17], in_data[16], in_data[15], // Row 4  
        in_data[16], in_data[15], in_data[14], in_data[13], in_data[12], in_data[11], // Row 5  
        in_data[12], in_data[11], in_data[10], in_data[9], in_data[8], in_data[7], // Row 6  
        in_data[8], in_data[7], in_data[6], in_data[5], in_data[4], in_data[3], // Row 7  
        in_data[4], in_data[3], in_data[2], in_data[1], in_data[0], in_data[31] // Row 8  
    );  
endmodule
```

Task 2.2 - Whitener (XOR)

After the expansion permutation, DES uses the XOR operation on the expanded right section and the round key. Note that both the right section and the keys are 48-bits in length. Also note that the round key is used only in this operation.

Create a module for XOR that implements this functionality.



```

module whitener_xor (
    input wire [47:0] expanded_data,
    input wire [47:0] round_key,
    output wire [47:0] xor_out
);

    assign xor_out = expanded_data ^ round_key;

endmodule

```

Task 2.3 - Construction of S-boxes.

DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output. The 48-bit data from the second operation is divided into eight 6-bit chunks, and each chunk is fed into a box. The result of each box is a 4-bit chunk; when these are combined the result is a 32-bit text.

```

module sboxes_combined (
    input wire [47:0] in_data,
    output wire [31:0] out_data
);

    sbox1 s1 (.in_chunk(in_data[47:42]), .out_chunk(out_data[31:28]));
    sbox2 s2 (.in_chunk(in_data[41:36]), .out_chunk(out_data[27:24]));
    sbox3 s3 (.in_chunk(in_data[35:30]), .out_chunk(out_data[23:20]));
    sbox4 s4 (.in_chunk(in_data[29:24]), .out_chunk(out_data[19:16]));
    sbox5 s5 (.in_chunk(in_data[23:18]), .out_chunk(out_data[15:12]));
    sbox6 s6 (.in_chunk(in_data[17:12]), .out_chunk(out_data[11:8]));
    sbox7 s7 (.in_chunk(in_data[11:6]), .out_chunk(out_data[7:4]));
    sbox8 s8 (.in_chunk(in_data[5:0]), .out_chunk(out_data[3:0]));

endmodule

```

1. Construct the table for S-box 1 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Table 1: S-box 1



```

module sbbox1 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
    wire [1:0] row = {in_chunk[5], in_chunk[0]};
    wire [3:0] col = in_chunk[4:1];
    reg [3:0] rom [0:3][0:15];
    initial begin
        rom[0][0]=14; rom[0][1]=4;  rom[0][2]=13; rom[0][3]=1;  rom[0][4]=2;  rom[0][5]=15; rom[0][6]=11; rom[0][7]=8;
        rom[0][8]=3;  rom[0][9]=10; rom[0][10]=6; rom[0][11]=12; rom[0][12]=5;  rom[0][13]=9;  rom[0][14]=0;  rom[0][15]=7;
        rom[1][0]=0;  rom[1][1]=15; rom[1][2]=7;  rom[1][3]=4;  rom[1][4]=14; rom[1][5]=2;  rom[1][6]=13; rom[1][7]=1;
        rom[1][8]=10; rom[1][9]=6;  rom[1][10]=12; rom[1][11]=11; rom[1][12]=9;  rom[1][13]=5;  rom[1][14]=3;  rom[1][15]=8;
        rom[2][0]=4;  rom[2][1]=1;  rom[2][2]=14; rom[2][3]=8;  rom[2][4]=13; rom[2][5]=6;  rom[2][6]=2;  rom[2][7]=11;
        rom[2][8]=15; rom[2][9]=12; rom[2][10]=9;  rom[2][11]=7;  rom[2][12]=3;  rom[2][13]=10; rom[2][14]=5;  rom[2][15]=0;
        rom[3][0]=15; rom[3][1]=12; rom[3][2]=8;  rom[3][3]=2;  rom[3][4]=4;  rom[3][5]=9;  rom[3][6]=1;  rom[3][7]=7;
        rom[3][8]=5;  rom[3][9]=11; rom[3][10]=3;  rom[3][11]=14; rom[3][12]=10; rom[3][13]=0;  rom[3][14]=6;  rom[3][15]=13;
    end
    assign out_chunk = rom[row][col];
endmodule

```

2. Construct the table for S-box 2 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	01	08	14	06	11	03	04	09	07	02	13	12	00	05	10
1	03	13	04	07	15	02	08	14	12	00	01	10	06	09	11	05
2	00	14	07	11	10	04	13	01	05	08	12	06	09	03	02	15
3	13	08	10	01	03	15	04	02	11	06	07	12	00	05	14	09

Table 2: S-box 2

```

module sbbox2 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
    wire [1:0] row = {in_chunk[5], in_chunk[0]};
    wire [3:0] col = in_chunk[4:1];
    reg [3:0] rom [0:3][0:15];
    initial begin
        rom[0][0]=15; rom[0][1]=1;  rom[0][2]=8;  rom[0][3]=14; rom[0][4]=6;  rom[0][5]=11; rom[0][6]=3;  rom[0][7]=4;
        rom[0][8]=9;  rom[0][9]=7;  rom[0][10]=2;  rom[0][11]=13; rom[0][12]=12; rom[0][13]=0;  rom[0][14]=5;  rom[0][15]=10;
        rom[1][0]=3;  rom[1][1]=13; rom[1][2]=4;  rom[1][3]=7;  rom[1][4]=15; rom[1][5]=2;  rom[1][6]=8;  rom[1][7]=14;
        rom[1][8]=12; rom[1][9]=0;  rom[1][10]=1;  rom[1][11]=10; rom[1][12]=6;  rom[1][13]=9;  rom[1][14]=11; rom[1][15]=5;
        rom[2][0]=0;  rom[2][1]=14; rom[2][2]=7;  rom[2][3]=11; rom[2][4]=10; rom[2][5]=4;  rom[2][6]=13; rom[2][7]=1;
        rom[2][8]=5;  rom[2][9]=8;  rom[2][10]=12; rom[2][11]=6;  rom[2][12]=9;  rom[2][13]=3;  rom[2][14]=2;  rom[2][15]=15;
        rom[3][0]=13; rom[3][1]=8;  rom[3][2]=10; rom[3][3]=1;  rom[3][4]=3;  rom[3][5]=15; rom[3][6]=4;  rom[3][7]=2;
        rom[3][8]=11; rom[3][9]=6;  rom[3][10]=7;  rom[3][11]=12; rom[3][12]=0;  rom[3][13]=5;  rom[3][14]=14; rom[3][15]=9;
    end
    assign out_chunk = rom[row][col];
endmodule

```

3. Construct the table for S-box 3 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	00	09	14	06	03	15	05	01	13	12	07	11	04	02	08
1	13	07	00	09	03	04	06	10	02	08	05	14	12	11	15	01
2	13	06	04	09	08	15	03	00	11	01	02	12	05	10	14	07
3	01	10	13	00	06	09	08	07	04	15	14	03	11	05	02	12

Table 3: S-box 4



```

module sbbox3 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
    wire [1:0] row = {in_chunk[5], in_chunk[0]};
    wire [3:0] col = in_chunk[4:1];
    reg [3:0] rom [0:3][0:15];
    initial begin
        rom[0][0]=10; rom[0][1]=0;  rom[0][2]=9;  rom[0][3]=14; rom[0][4]=6;  rom[0][5]=3;  rom[0][6]=15; rom[0][7]=5;
        rom[0][8]=1;  rom[0][9]=13; rom[0][10]=12; rom[0][11]=7;  rom[0][12]=11; rom[0][13]=4;  rom[0][14]=2;  rom[0][15]=8;
        rom[1][0]=13; rom[1][1]=7;  rom[1][2]=0;  rom[1][3]=9;  rom[1][4]=3;  rom[1][5]=4;  rom[1][6]=6;  rom[1][7]=10;
        rom[1][8]=2;  rom[1][9]=8;  rom[1][10]=5;  rom[1][11]=14; rom[1][12]=12; rom[1][13]=11; rom[1][14]=15; rom[1][15]=1;
        rom[2][0]=13; rom[2][1]=6;  rom[2][2]=4;  rom[2][3]=9;  rom[2][4]=8;  rom[2][5]=15; rom[2][6]=3;  rom[2][7]=0;
        rom[2][8]=11; rom[2][9]=1;  rom[2][10]=2;  rom[2][11]=12; rom[2][12]=5;  rom[2][13]=10; rom[2][14]=14; rom[2][15]=7;
        rom[3][0]=1;  rom[3][1]=10; rom[3][2]=13; rom[3][3]=0;  rom[3][4]=6;  rom[3][5]=9;  rom[3][6]=8;  rom[3][7]=7;
        rom[3][8]=4;  rom[3][9]=15; rom[3][10]=14; rom[3][11]=3;  rom[3][12]=11; rom[3][13]=5;  rom[3][14]=2;  rom[3][15]=12;
    end
    assign out_chunk = rom[row][col];
endmodule

```

4. Construct the table for S-box 4 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	07	13	14	03	00	6	09	10	1	02	08	05	11	12	04	15
1	13	08	11	05	06	15	00	03	04	07	02	12	01	10	14	09
2	10	06	09	00	12	11	07	13	15	01	03	14	05	02	08	04
3	03	15	00	06	10	01	13	08	09	04	05	11	12	07	02	14

Table 4: S-box 4

```

module sbbox4 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
    wire [1:0] row = {in_chunk[5], in_chunk[0]};
    wire [3:0] col = in_chunk[4:1];
    reg [3:0] rom [0:3][0:15];
    initial begin
        rom[0][0]=7;  rom[0][1]=13; rom[0][2]=14; rom[0][3]=3;  rom[0][4]=0;  rom[0][5]=6;  rom[0][6]=9;  rom[0][7]=10;
        rom[0][8]=1;  rom[0][9]=2;  rom[0][10]=8;  rom[0][11]=5;  rom[0][12]=11; rom[0][13]=12; rom[0][14]=4;  rom[0][15]=15;
        rom[1][0]=13; rom[1][1]=8;  rom[1][2]=11; rom[1][3]=5;  rom[1][4]=6;  rom[1][5]=15; rom[1][6]=0;  rom[1][7]=3;
        rom[1][8]=4;  rom[1][9]=7;  rom[1][10]=2;  rom[1][11]=12; rom[1][12]=1;  rom[1][13]=10; rom[1][14]=14; rom[1][15]=9;
        rom[2][0]=10; rom[2][1]=6;  rom[2][2]=9;  rom[2][3]=0;  rom[2][4]=12; rom[2][5]=11; rom[2][6]=7;  rom[2][7]=13;
        rom[2][8]=15; rom[2][9]=1;  rom[2][10]=3;  rom[2][11]=14; rom[2][12]=5;  rom[2][13]=2;  rom[2][14]=8;  rom[2][15]=4;
        rom[3][0]=3;  rom[3][1]=15; rom[3][2]=0;  rom[3][3]=6;  rom[3][4]=10; rom[3][5]=1;  rom[3][6]=13; rom[3][7]=8;
        rom[3][8]=9;  rom[3][9]=4;  rom[3][10]=5;  rom[3][11]=11; rom[3][12]=12; rom[3][13]=7;  rom[3][14]=2;  rom[3][15]=14;
    end
    assign out_chunk = rom[row][col];
endmodule

```

5. Construct the table for S-box 5 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	02	12	04	01	07	10	11	06	08	05	03	15	13	00	14	09
1	14	11	02	12	04	07	13	01	05	00	15	10	03	09	08	06
2	04	02	01	11	10	13	07	08	15	09	12	05	06	03	00	14
3	11	08	12	07	01	14	02	13	06	15	00	09	10	04	05	03

Table 5: S-box 5



```

module sbx5 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
  wire [1:0] row = {in_chunk[5], in_chunk[0]};
  wire [3:0] col = in_chunk[4:1];
  reg [3:0] rom [0:3][0:15];
  initial begin
    rom[0][0]=2; rom[0][1]=12; rom[0][2]=4; rom[0][3]=1; rom[0][4]=7; rom[0][5]=10; rom[0][6]=11; rom[0][7]=6;
    rom[0][8]=8; rom[0][9]=5; rom[0][10]=3; rom[0][11]=15; rom[0][12]=13; rom[0][13]=0; rom[0][14]=14; rom[0][15]=9;
    rom[1][0]=14; rom[1][1]=11; rom[1][2]=2; rom[1][3]=12; rom[1][4]=4; rom[1][5]=7; rom[1][6]=13; rom[1][7]=1;
    rom[1][8]=5; rom[1][9]=0; rom[1][10]=15; rom[1][11]=10; rom[1][12]=3; rom[1][13]=9; rom[1][14]=8; rom[1][15]=6;
    rom[2][0]=4; rom[2][1]=2; rom[2][2]=1; rom[2][3]=11; rom[2][4]=10; rom[2][5]=13; rom[2][6]=7; rom[2][7]=8;
    rom[2][8]=15; rom[2][9]=9; rom[2][10]=12; rom[2][11]=5; rom[2][12]=6; rom[2][13]=3; rom[2][14]=0; rom[2][15]=14;
    rom[3][0]=11; rom[3][1]=8; rom[3][2]=12; rom[3][3]=7; rom[3][4]=1; rom[3][5]=14; rom[3][6]=2; rom[3][7]=13;
    rom[3][8]=6; rom[3][9]=15; rom[3][10]=0; rom[3][11]=9; rom[3][12]=10; rom[3][13]=4; rom[3][14]=5; rom[3][15]=3;
  end
  assign out_chunk = rom[row][col];
endmodule

```

6. Construct the table for S-box 6 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	01	10	15	09	02	06	08	00	13	03	04	14	07	05	11
1	10	15	04	02	07	12	09	05	06	01	13	14	00	11	03	08
2	09	14	15	05	02	08	12	03	07	00	04	10	01	13	11	06
3	04	03	02	12	09	05	15	10	11	14	01	07	10	00	08	13

Table 6: S-box 6

```

module sbx6 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
  wire [1:0] row = {in_chunk[5], in_chunk[0]};
  wire [3:0] col = in_chunk[4:1];
  reg [3:0] rom [0:3][0:15];
  initial begin
    rom[0][0]=12; rom[0][1]=1; rom[0][2]=10; rom[0][3]=15; rom[0][4]=9; rom[0][5]=2; rom[0][6]=6; rom[0][7]=8;
    rom[0][8]=0; rom[0][9]=13; rom[0][10]=3; rom[0][11]=4; rom[0][12]=14; rom[0][13]=7; rom[0][14]=5; rom[0][15]=11;
    rom[1][0]=10; rom[1][1]=15; rom[1][2]=4; rom[1][3]=2; rom[1][4]=7; rom[1][5]=12; rom[1][6]=9; rom[1][7]=5;
    rom[1][8]=6; rom[1][9]=1; rom[1][10]=13; rom[1][11]=14; rom[1][12]=0; rom[1][13]=11; rom[1][14]=3; rom[1][15]=8;
    rom[2][0]=9; rom[2][1]=14; rom[2][2]=15; rom[2][3]=5; rom[2][4]=2; rom[2][5]=8; rom[2][6]=12; rom[2][7]=3;
    rom[2][8]=7; rom[2][9]=0; rom[2][10]=4; rom[2][11]=10; rom[2][12]=1; rom[2][13]=13; rom[2][14]=11; rom[2][15]=6;
    rom[3][0]=4; rom[3][1]=3; rom[3][2]=2; rom[3][3]=12; rom[3][4]=9; rom[3][5]=5; rom[3][6]=15; rom[3][7]=10;
    rom[3][8]=11; rom[3][9]=14; rom[3][10]=1; rom[3][11]=7; rom[3][12]=10; rom[3][13]=0; rom[3][14]=8; rom[3][15]=13;
  end
  assign out_chunk = rom[row][col];
endmodule

```

7. Construct the table for S-box 7 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	00	08	13	03	12	09	07	05	10	06	01
1	13	00	11	07	04	09	01	10	14	03	05	12	02	15	08	06
2	01	04	11	13	12	03	07	14	10	15	06	08	00	05	09	02
3	06	11	13	08	01	04	10	07	09	05	00	15	14	02	03	12

Table 7: S-box 7



```

module sbbox7 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
  wire [1:0] row = {in_chunk[5], in_chunk[0]};
  wire [3:0] col = in_chunk[4:1];
  reg [3:0] rom [0:3][0:15];
  initial begin
    rom[0][0]=4; rom[0][1]=11; rom[0][2]=2; rom[0][3]=14; rom[0][4]=15; rom[0][5]=0; rom[0][6]=8; rom[0][7]=13;
    rom[0][8]=3; rom[0][9]=12; rom[0][10]=9; rom[0][11]=7; rom[0][12]=5; rom[0][13]=10; rom[0][14]=6; rom[0][15]=1;
    rom[1][0]=13; rom[1][1]=0; rom[1][2]=11; rom[1][3]=7; rom[1][4]=4; rom[1][5]=9; rom[1][6]=1; rom[1][7]=10;
    rom[1][8]=14; rom[1][9]=3; rom[1][10]=5; rom[1][11]=12; rom[1][12]=2; rom[1][13]=15; rom[1][14]=8; rom[1][15]=6;
    rom[2][0]=1; rom[2][1]=4; rom[2][2]=11; rom[2][3]=13; rom[2][4]=12; rom[2][5]=3; rom[2][6]=7; rom[2][7]=14;
    rom[2][8]=10; rom[2][9]=15; rom[2][10]=6; rom[2][11]=8; rom[2][12]=0; rom[2][13]=5; rom[2][14]=9; rom[2][15]=2;
    rom[3][0]=6; rom[3][1]=11; rom[3][2]=13; rom[3][3]=8; rom[3][4]=1; rom[3][5]=4; rom[3][6]=10; rom[3][7]=7;
    rom[3][8]=9; rom[3][9]=5; rom[3][10]=0; rom[3][11]=15; rom[3][12]=14; rom[3][13]=2; rom[3][14]=3; rom[3][15]=12;
  end
  assign out_chunk = rom[row][col];
endmodule

```

8. Construct the table for S-box 8 using the table below.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	02	08	04	06	15	11	01	10	09	03	14	05	00	12	07
1	01	15	13	08	10	03	07	04	12	05	06	11	10	14	09	02
2	07	11	04	01	09	12	14	02	00	06	10	10	15	03	05	08
3	02	01	14	07	04	10	8	13	15	12	09	09	03	05	06	11

Table 8: S-box 8

```

module sbbox8 (input wire [5:0] in_chunk, output wire [3:0] out_chunk);
  wire [1:0] row = {in_chunk[5], in_chunk[0]};
  wire [3:0] col = in_chunk[4:1];
  reg [3:0] rom [0:3][0:15];
  initial begin
    rom[0][0]=13; rom[0][1]=2; rom[0][2]=8; rom[0][3]=4; rom[0][4]=6; rom[0][5]=15; rom[0][6]=11; rom[0][7]=1;
    rom[0][8]=10; rom[0][9]=9; rom[0][10]=3; rom[0][11]=14; rom[0][12]=5; rom[0][13]=0; rom[0][14]=12; rom[0][15]=7;
    rom[1][0]=1; rom[1][1]=15; rom[1][2]=13; rom[1][3]=8; rom[1][4]=10; rom[1][5]=3; rom[1][6]=7; rom[1][7]=4;
    rom[1][8]=12; rom[1][9]=5; rom[1][10]=6; rom[1][11]=11; rom[1][12]=0; rom[1][13]=14; rom[1][14]=9; rom[1][15]=2;
    rom[2][0]=7; rom[2][1]=11; rom[2][2]=4; rom[2][3]=1; rom[2][4]=9; rom[2][5]=12; rom[2][6]=14; rom[2][7]=2;
    rom[2][8]=0; rom[2][9]=6; rom[2][10]=10; rom[2][11]=13; rom[2][12]=15; rom[2][13]=3; rom[2][14]=5; rom[2][15]=8;
    rom[3][0]=2; rom[3][1]=1; rom[3][2]=14; rom[3][3]=7; rom[3][4]=4; rom[3][5]=10; rom[3][6]=8; rom[3][7]=13;
    rom[3][8]=15; rom[3][9]=12; rom[3][10]=9; rom[3][11]=0; rom[3][12]=3; rom[3][13]=5; rom[3][14]=6; rom[3][15]=11;
  end
  assign out_chunk = rom[row][col];
endmodule

```

Task 2.4 - Performing Straight Permutation

Create a module that takes as input a 32-bit text and returns another 32-bit text. The input/output relationship follows the table below. For example, the seventh bit of the input becomes the second bit of the output.



16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Figure 4: Straight Permutation Table

```
module straight_permutation (
    input wire [31:0] in_data,
    output wire [31:0] out_data
);

    assign out_data = {
        in_data[16], in_data[25], in_data[12], in_data[11], in_data[3], in_data[20], in_data[4], in_data[15], // Row 1
        in_data[31], in_data[17], in_data[9], in_data[6], in_data[27], in_data[14], in_data[1], in_data[22], // Row 2
        in_data[30], in_data[24], in_data[8], in_data[18], in_data[0], in_data[5], in_data[29], in_data[23], // Row 3
        in_data[13], in_data[19], in_data[2], in_data[26], in_data[10], in_data[21], in_data[28], in_data[7] // Row 4
    };

endmodule
```

Task 2.5 - Designing the DES function

Integrate the functions previously designed in this task to assemble the complete DES function, mirroring the structure illustrated in Figure 2.



```
////////////////////////////////////  
module des_function (  
    input wire [31:0] in_data,  
    input wire [47:0] round_key,  
    output wire [31:0] out_data  
);  
    wire [47:0] expanded_data;  
    wire [47:0] xor_data;  
    wire [31:0] sbbox_data;  
    expansion_p_box ep_box (  
        .in_data(in_data),  
        .out_data(expanded_data)  
    );  
  
    whitener_xor xor_step (  
        .expanded_data(expanded_data),  
        .round_key(round_key),  
        .xor_out(xor_data)  
    );  
  
    sbboxes_combined s_boxes (  
        .in_data(xor_data),  
        .out_data(sbbox_data)  
    );  
  
    straight_permutation sp_box (  
        .in_data(sbbox_data),  
        .out_data(out_data)  
    );  
  
endmodule
```

Task 3 – Creating the DES Cipher

Using mixers and swappers, we can create the cipher and reverse cipher, each having 16 rounds. The cipher is used at the encryption site; the reverse cipher is used at the decryption site. In this lab, we will only be creating the cipher for encryption.

Note that there are two approaches available for constructing the cipher in your DES implementation. In the first approach, illustrated in Figure 5, the final round does not include a swapper. Alternatively, you can opt for the second approach, where all rounds are kept uniform, and an additional swapper is introduced at the end to counteract the effect of the swapper in the 16th round. You may choose either approach for your DES implementation.

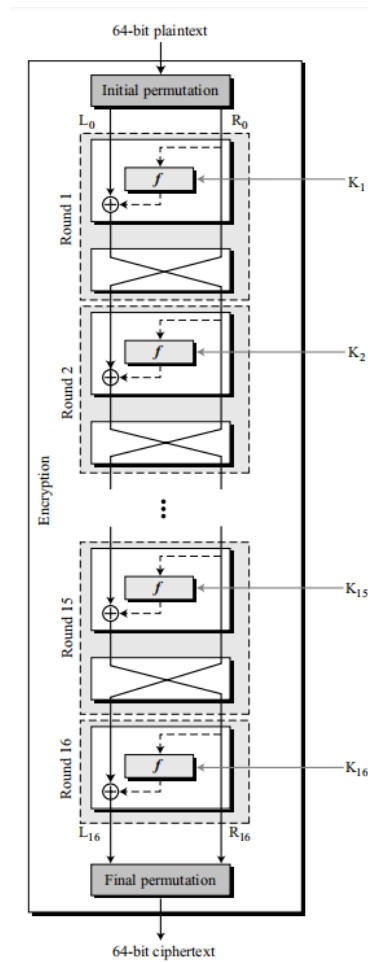


Figure 5: First Approach

Below is the pseudocode for the DES cipher:



```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round!=16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}

mixer (leftBlock[32], rightBlock[32], RoundKey[48])
{
    copy (32, rightBlock, T1)
    function (T1, RoundKey, T2)
    exclusiveOr (32, leftBlock, T2, T3)
    copy (32, T3, rightBlock)
}

swapper (leftBlock[32], rightBlock[32])
{
    copy (32, leftBlock, T)
    copy (32, rightBlock, leftBlock)
    copy (32, T, rightBlock)
}

function (inBlock[32], RoundKey[48], outBlock[32])
{
    permute (32, 48, inBlock, T1, ExpansionPermutationTable)
    exclusiveOr (48, T1, RoundKey, T2)
    substitute (T2, T3, SubstitutionTables)
    permute (32, 32, T3, outBlock, StraightPermutationTable)
}

substitute (inBlock[32], outBlock[48], SubstitutionTables[8, 4, 16])
{
    for (i = 1 to 8)
    {
        row  $\leftarrow 2 \times \text{inBlock}[i \times 6 + 1] + \text{inBlock}[i \times 6 + 6]$ 
        col  $\leftarrow 8 \times \text{inBlock}[i \times 6 + 2] + 4 \times \text{inBlock}[i \times 6 + 3] +$ 
             $2 \times \text{inBlock}[i \times 6 + 4] + \text{inBlock}[i \times 6 + 5]$ 

        value = SubstitutionTables [i][row][col]

        outBlock[[i × 4 + 1]  $\leftarrow$  value / 8;      value  $\leftarrow$  value mod 8
        outBlock[[i × 4 + 2]  $\leftarrow$  value / 4;      value  $\leftarrow$  value mod 4
        outBlock[[i × 4 + 3]  $\leftarrow$  value / 2;      value  $\leftarrow$  value mod 2
        outBlock[[i × 4 + 4]  $\leftarrow$  value
    }
}
```

Figure 6: DES Algorithm



```
module des_cipher (
    input wire [63:0] plaintext,
    input wire [767:0] round_keys_flat,
    output wire [63:0] ciphertext
);
    wire [63:0] ip_out;
    initial_permutation ip_inst (
        .in_data(plaintext),
        .out_data(ip_out)
    );
    wire [31:0] L [0:16];
    wire [31:0] R [0:16];
    assign L[0] = ip_out[63:32];
    assign R[0] = ip_out[31:0];
    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : rounds
            des_round round_inst (
                .L_in(L[i]),
                .R_in(R[i]),
                .round_key(round_keys_flat[767 - (i*48) : 720 - (i*48)]),
                .L_out(L[i+1]),
                .R_out(R[i+1])
            );
        end
    endgenerate
    wire [63:0] pre_fp_data;
    assign pre_fp_data = {R[16], L[16]};
    final_permutation fp_inst (
        .in_data(pre_fp_data),
        .out_data(ciphertext)
    );
endmodule
```

Task 4 – Key Generation

The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key. However, the cipher key is normally given as a 64-bit key in which 8 extra bits are the parity bits, which are dropped before the actual key-generation process.

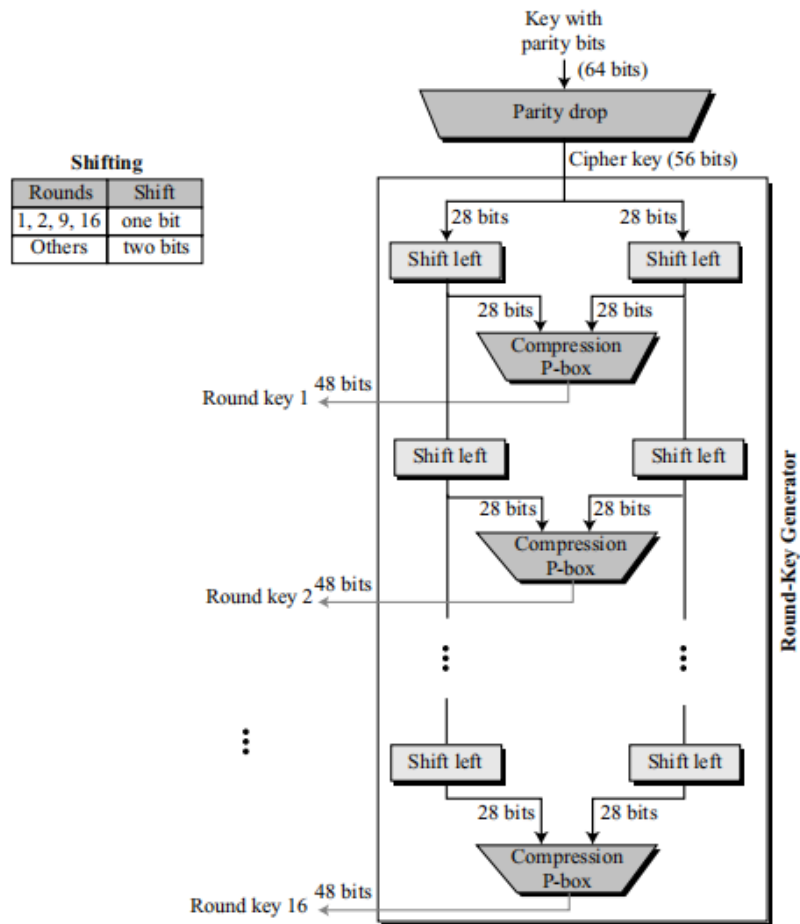


Figure 7: Key Generation

Task 4.1 - Constructing the P-Box.

Create a function that takes 64 bits as input and uses the parity drop table to return a 56-bit cipher key. The parity drop permutation table is given below.

57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	38
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04



Figure 8: Parity Drop Table

```
module parity_drop (  
    input wire [63:0] in_key,  
    output wire [55:0] out_key  
);  
  
    assign out_key = (  
        in_key[7], in_key[15], in_key[23], in_key[31], in_key[39], in_key[47], in_key[55], in_key[63], // Row 1  
        in_key[6], in_key[14], in_key[22], in_key[30], in_key[38], in_key[46], in_key[54], in_key[62], // Row 2  
        in_key[5], in_key[13], in_key[21], in_key[29], in_key[37], in_key[45], in_key[53], in_key[61], // Row 3  
        in_key[4], in_key[12], in_key[20], in_key[28], in_key[1], in_key[9], in_key[17], in_key[25], // Row 4  
        in_key[33], in_key[41], in_key[49], in_key[57], in_key[2], in_key[10], in_key[18], in_key[26], // Row 5  
        in_key[34], in_key[42], in_key[50], in_key[58], in_key[3], in_key[11], in_key[19], in_key[27], // Row 6  
        in_key[35], in_key[43], in_key[51], in_key[59], in_key[36], in_key[44], in_key[52], in_key[60] // Row 7  
    );  
  
endmodule
```

Task 4.2 - Constructing the shift-left function.

After the straight permutation, the key is divided into two 28-bit parts. Each part is shifted left (circular shift) one or two bits. In rounds 1, 2, 9, and 16, shifting is one bit; in the other rounds, it is two bits. The two parts are then combined to form a 56-bit part.

Design a function that employs the algorithm depicted in Figure 9. This function should accept a 28-bit block and the specific number of shifts needed, returning the resulting 28-bit block. Refer to Table 9 for the exact number of shifts required for each round.

```
shiftLeft (block[28], numOfShifts)  
{  
    for (i = 1 to numOfShifts)  
    {  
        T ← block[1]  
        for (j = 2 to 28)  
        {  
            block [j-1] ← block [j]  
        }  
        block[28] ← T  
    }  
}
```

Figure 9: Shift Left Algorithm

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Table 9: Number of bits shifts for each round



```
module shift_left (  
    input wire [27:0] in_block,  
    input wire [1:0] num_shifts,  
    output wire [27:0] out_block  
);  
  
    // Perform a circular left shift based on num_shifts (1 or 2)  
    assign out_block = (num_shifts == 1) ? {in_block[26:0], in_block[27]} :  
        {in_block[25:0], in_block[27:26]};  
  
endmodule
```

Task 4.3 - Key Compression Table.

The compression permutation (P-box) changes the 58 bits to 48 bits, which are used as a key for a round.

Create a module that accepts a 58-bit input and based on the provided permutation table, generates a 48-bit output.

14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Figure 10: Key Compression Table

```
module key_compression (  
    input wire [55:0] in_key,  
    output wire [47:0] out_key  
);  
  
    assign out_key = {  
        in_key[42], in_key[39], in_key[45], in_key[32], in_key[55], in_key[51], in_key[53], in_key[28], // Row 1  
        in_key[41], in_key[50], in_key[35], in_key[46], in_key[33], in_key[37], in_key[44], in_key[52], // Row 2  
        in_key[30], in_key[48], in_key[40], in_key[49], in_key[29], in_key[36], in_key[43], in_key[54], // Row 3  
        in_key[15], in_key[4], in_key[25], in_key[19], in_key[9], in_key[1], in_key[26], in_key[16], // Row 4  
        in_key[5], in_key[11], in_key[23], in_key[8], in_key[12], in_key[7], in_key[17], in_key[0], // Row 5  
        in_key[22], in_key[3], in_key[10], in_key[14], in_key[6], in_key[20], in_key[27], in_key[24] // Row 6  
    };  
  
endmodule
```

Task 4.4 - Designing the Key Generator.



Create a function that generates the keys required for all 16 rounds following the algorithm presented in Figure 11. To better grasp the operation of this function, you can also refer to Figure 7.

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
    permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
    split (56, 28, cipherKey, leftKey, rightKey)
    for (round = 1 to 16)
    {
        shiftLeft (leftKey, ShiftTable[round])
        shiftLeft (rightKey, ShiftTable[round])
        combine (28, 56, leftKey, rightKey, preRoundKey)
        permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
    }
}
```

Figure 11: Round Key Generator Algorithm

```
module key_generator (
    input wire [63:0] key_with_parities,
    output wire [767:0] round_keys_flat
);

    wire [55:0] cipher_key;
    parity_drop pd_inst (
        .in_key(key_with_parities),
        .out_key(cipher_key)
    );

    wire [27:0] L [0:16];
    wire [27:0] R [0:16];
    assign L[0] = cipher_key[55:28];
    assign R[0] = cipher_key[27:0];
    wire [1:0] shift_table [0:15];
    assign shift_table[0] = 1; assign shift_table[1] = 1; assign shift_table[2] = 2; assign shift_table[3] = 2;
    assign shift_table[4] = 2; assign shift_table[5] = 2; assign shift_table[6] = 2; assign shift_table[7] = 2;
    assign shift_table[8] = 1; assign shift_table[9] = 2; assign shift_table[10] = 2; assign shift_table[11] = 2;
    assign shift_table[12] = 2; assign shift_table[13] = 2; assign shift_table[14] = 2; assign shift_table[15] = 1;
```



```
genvar i;
generate
  for (i = 0; i < 16; i = i + 1) begin : key_rounds
    shift_left sl_left (
      .in_block(L[i]),
      .num_shifts(shift_table[i]),
      .out_block(L[i+1])
    );
    shift_left sl_right (
      .in_block(R[i]),
      .num_shifts(shift_table[i]),
      .out_block(R[i+1])
    );
    key_compression kc_inst (
      .in_key({L[i+1], R[i+1]}),
      .out_key(round_keys_flat[767 - (i*48) : 720 - (i*48)])
    );
  end
end
endgenerate
endmodule
```

Task 5 – Validating the DES cipher.

1. Run the key generator function for “AABB09182736CCDD” to obtain the round keys for all 16 rounds.
2. Run the DES cipher for “123456ABCD132536”.
3. Verify your output. Your results should match the values shown in Figure 12.



```
module tb_des();

    reg [63:0] plaintext;
    reg [63:0] key;
    wire [767:0] round_keys;
    wire [63:0] ciphertext;
    key_generator uut_key_gen (
        .key_with_parities(key),
        .round_keys_flat(round_keys)
    );
    des_cipher uut_cipher (
        .plaintext(plaintext),
        .round_keys_flat(round_keys),
        .ciphertext(ciphertext)
    );
    initial begin
        key = 64'hAABB09182736CCDD;
        plaintext = 64'h123456ABCD132536;
        #100;
        $display("=====");
        $display("Input (Plaintext):  %h", plaintext);
        $display("Key (Input Key):    %h", key);
        $display("Round Keys (All 16): %h", round_keys);
        $display("After combination:  %h%h", uut_cipher.R[16], uut_cipher.L[16]);
        $display("Output (Ciphertext): %h", ciphertext);
        $display("=====");

        $finish;
    end
endmodule
```

Name	Value	99,995 ps	99,996 ps	99,997 ps	99,998 ps	99,999 ps
> plaintext[63:0]	123456abcd132536	123456abcd132536				
> key[63:0]	aabb09182736ccdd	aabb09182736ccdd				
> round_keys[767:0]	194cd072de8c4568581abccc06eda4acf5b5da2d032b6ee369a629fec913c1948e870	194cd072de8c4568581abccc06eda4acf5b5da2d032b6ee369a629fec913c1948e870				
> ciphertext[63:0]	c0b7a8d05f3a829c	c0b7a8d05f3a829c				

Note: For step 1 and 2, you will have to convert the plaintext into binary first.



<i>Plaintext:</i> 123456ABCD132536			
<i>After initial permutation:</i> 14A7D67818CA18AD <i>After splitting:</i> L ₀ =14A7D678 R ₀ =18CA18AD			
<i>Round</i>	<i>Left</i>	<i>Right</i>	<i>Round Key</i>
<i>Round 1</i>	18CA18AD	5A78E394	194CD072DE8C
<i>Round 2</i>	5A78E394	4A1210F6	4568581ABCCE
<i>Round 3</i>	4A1210F6	B8089591	06EDA4ACF5B5
<i>Round 4</i>	B8089591	236779C2	DA2D032B6EE3
<i>Round 5</i>	236779C2	A15A4B87	69A629FEC913
<i>Round 6</i>	A15A4B87	2E8F9C65	C1948E87475E
<i>Round 7</i>	2E8F9C65	A9FC20A3	708AD2DDB3C0
<i>Round 8</i>	A9FC20A3	308BEE97	34F822F0C66D
<i>Round 9</i>	308BEE97	10AF9D37	84BB4473DCCC
<i>Round 10</i>	10AF9D37	6CA6CB20	02765708B5BF
<i>Round 11</i>	6CA6CB20	FF3C485F	6D5560AF7CA5
<i>Round 12</i>	FF3C485F	22A5963B	C2C1E96A4BF3
<i>Round 13</i>	22A5963B	387CCDAA	99C31397C91F
<i>Round 14</i>	387CCDAA	BD2DD2AB	251B8BC717D0
<i>Round 15</i>	BD2DD2AB	CF26B472	3330C5D9A36D
<i>Round 16</i>	19BA9212	CF26B472	181C5D75C66D
<i>After combination:</i> 19BA9212CF26B472			
<i>Ciphertext:</i> C0B7A8D05F3A829C		<i>(after final permutation)</i>	

Figure 12: Trace of Data



Assessment Rubric
Lab 10
Data Encryption Standard (DES)

Name:	Student ID:
--------------	--------------------

Points Distribution

Task No.	LR 2 Code	LR5 Results	LR9 Report
Task 1	5		
Task 2.1	5		
Task 2.2	5		
Task 2.3	5		
Task 2.4	5		
Task 2.5	5		
Task 3	15		
Task 4.1	5		
Task 4.2	5		
Task 4.3	5		
Task 4.4	10		
Task 5	5	10	
Total	/75	/10	/5
CLO Mapped	CLO 2	CLO 2	CLO2

Affective Domain Rubric		Points	CLO Mapped
AR 7	Report Submission	/10	CLO 2

CLO	Total Points	Points Obtained
2	90	
2	10	
Total	100	



For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

Lab Evaluation Assessment Rubric

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.